# User Stories

ShopSense: the personalised shopping research agent

| | |
|---|---|
| **Document** | User Stories |
| **Project** | ShopSense: the personalised shopping research agent |
| **Course** | IT314 Software Engineering |
| **Team** | Group 13 (10 members) |
| **Version** | 1.0 |
| **Date** | 14 October 2026 |

This document is the product backlog of ShopSense. It holds all 51 user stories, grouped by epic. Each story says who wants something, what they want and why, followed by the acceptance criteria that must pass before the story is complete.

## 1. How to read a user story

- **Story statement.** Written as "As a *type of user*, I want *a goal*, so that *a benefit*."
- **Acceptance criteria.** Clear checks that must all pass before the story is accepted at the sprint review.
- **Priority.** Uses the MoSCoW method: **Must** (the product cannot work without it), **Should** (important, but the product still works without it) and **Could** (useful, done if time allows).
- **Story points.** The relative size of the work. A larger number means more work.
- **Sprint.** The two-week sprint in which the story is planned (Sprint 1 to Sprint 6).
- **Tier. Core** stories are committed. **Plus** stories are the first to move to a later sprint if a sprint runs late. The **Stretch** story is done only if time allows.
- **Requirements.** The requirements from the Requirements Specification that the story builds.

### Backlog summary

| Tier | Stories | Story points |
|---|---|---|
| Core | 45 | 216 |
| Plus | 5 | 15 |
| Stretch | 1 | 8 |
| **Total** | **51** | **239** |

By priority: 35 Must, 12 Should and 4 Could.

### Epics

| Epic | Name and purpose | Stories | Points |
|---|---|---|---|
| Epic 1 | **User Discovery and Personalization**<br>Understand the shopper: intent, criteria, interview, memory, rules and exclusions. | 12 | 58 |
| Epic 2 | **Multi-Source Research and Retrieval**<br>Find and collect evidence from several sources, and read community posts well. | 5 | 29 |
| Epic 3 | **Evidence, Product Intelligence and Verification**<br>Extract claims, match products, verify, flag conflicts and drop unsourced products. | 4 | 26 |
| Epic 4 | **Personalized Recommendation and Explainability**<br>Enrich, filter, rank, explain, cite and compare. | 9 | 45 |
| Epic 5 | **Commerce and Purchase Discovery**<br>Price, availability and buy links for the exact model. | 1 | 5 |
| Epic 6 | **Platform Engineering, Security and Reliability**<br>A skeleton that always runs: secure, reliable, fast, modular and tested. | 20 | 76 |

## 2. List of user stories

| ID | Title | Priority | Points | Sprint | Tier |
|---|---|---|---|---|---|
| **Epic 1: User Discovery and Personalization** | | | | | |
| **US-01** | Describe the need in plain words | Must | 5 | 1 | Core |
| **US-02** | Detect category, use, budget and rules | Must | 5 | 1 | Core |
| **US-03** | Generate category-specific criteria | Must | 5 | 1 | Core |
| **US-04** | Smart interview that asks only what matters | Should | 8 | 4 | Core |
| **US-05** | One question at a time, adapted to earlier answers | Must | 8 | 1 | Core |
| **US-06** | Remember preferences across searches | Should | 8 | 5 | Core |
| **US-07** | Explain unfamiliar terms on request | Should | 2 | 6 | Core |
| **US-08** | Understand rough answers like "medium" | Must | 3 | 2 | Core |
| **US-09** | Keep must-haves separate from preferences | Must | 5 | 3 | Core |
| **US-10** | Rule out brands and features | Must | 3 | 3 | Core |
| **US-11** | See and control what the app remembers | Should | 3 | 6 | Core |
| **US-39** | Understand questions, skips and off-topic replies in the interview | Must | 3 | 4 | Core |
| **Epic 2: Multi-Source Research and Retrieval** | | | | | |
| **US-12** | Research several sources in parallel | Must | 8 | 2 | Core |
| **US-13** | Score Reddit quality and count recommenders exactly | Should | 3 | 2 | Core |
| **US-36** | Classifier for promotional and duplicate comments | Could | 8 | 6 | Stretch |
| **US-38** | Add YouTube and hard-to-read pages | Must | 5 | 5 | Core |
| **US-42** | Read community posts like a person would | Should | 5 | 5 | Plus |
| **Epic 3: Evidence, Product Intelligence and Verification** | | | | | |
| **US-14** | Extract claims with exact quotes | Must | 8 | 2 | Core |

| ID | Title | Priority | Points | Sprint | Tier |
|---|---|---|---|---|---|
| **US-15** | Match product names and variants | Must | 8 | 3 | Core |
| **US-16** | Learned source trust, disputes and unknowns | Must | 8 | 3 | Core |
| **US-40** | Never recommend an unsourced product | Must | 2 | 3 | Core |
| **Epic 4: Personalized Recommendation and Explainability** | | | | | |
| **US-17** | Filter out products that break the rules | Must | 3 | 3 | Core |
| **US-18** | Rank on the user's priorities with coverage | Must | 8 | 3 | Core |
| **US-19** | Honest confidence and ties from simulations | Should | 5 | 4 | Core |
| **US-20** | Explain why a product fits | Must | 5 | 4 | Core |
| **US-21** | Click any claim to see its source | Must | 3 | 3 | Core |
| **US-22** | Check every explanation sentence against its evidence | Should | 5 | 5 | Core |
| **US-23** | Compare products and re-rank instantly | Must | 8 | 5 | Core |
| **US-37** | Save or share results | Could | 3 | 6 | Plus |
| **US-41** | Look up missing facts for the top products | Must | 5 | 5 | Core |
| **Epic 5: Commerce and Purchase Discovery** | | | | | |
| **US-24** | Current price and buy link for the exact model | Must | 5 | 5 | Core |
| **Epic 6: Platform Engineering, Security and Reliability** | | | | | |
| **US-25** | Repository, CI and secret scanning | Must | 3 | 1 | Core |
| **US-26** | One LLM wrapper with fallback and context fitting | Must | 5 | 1 | Core |
| **US-27** | Fetch sources in parallel | Must | 2 | 2 | Core |
| **US-28** | Treat retrieved content as untrusted | Must | 3 | 2 | Core |
| **US-29** | Trace every claim to its evidence | Must | 3 | 6 | Core |
| **US-30** | Login and private preferences | Must | 5 | 4 | Core |
| **US-31** | Handle outside failures gracefully | Must | 5 | 4 | Core |
| **US-32** | Core logic testable offline | Must | 3 | 2 | Core |
| **US-33** | Plain language and accessible controls | Must | 5 | 6 | Core |
| **US-34** | Fast responses and exact-match cache | Should | 5 | 6 | Core |
| **US-35** | Modules with clear interfaces | Should | 3 | 6 | Core |
| **US-46** | Circuit breaker and provider rotation | Must | 4 | 2 | Core |
| **US-47** | Per-user rate limits | Should | 2 | 6 | Plus |
| **US-48** | Golden tests for broken LLM output | Must | 3 | 2 | Core |
| **US-49** | Recorded-run replay and the CI quality gate | Must | 5 | 4 | Core |
| **US-50** | Worst-case test suite | Must | 5 | 6 | Core |
| **US-51** | Measure consistency, personalisation and popularity bias | Should | 5 | 6 | Core |
| **US-52** | Walking skeleton, pipeline runner and CLI | Must | 5 | 1 | Core |
| **US-53** | Progress survives a dropped connection | Could | 2 | 6 | Plus |
| **US-54** | Team diagnostics page | Could | 3 | 6 | Plus |

# 3. User stories in full

## Epic 1: User Discovery and Personalization

Understand the shopper: intent, criteria, interview, memory, rules and exclusions.

**US-01: Describe the need in plain words**

Priority: **Must** | Story points: **5** | Sprint: **1** | Tier: **Core** | Requirements: **FR-01**

**As a** shopper, **I want** to describe what I need in my own words, **so that** I do not have to fill a form or know product jargon.

**Acceptance criteria**

1. One text box accepts up to 1,000 characters, and nothing else is needed to start
2. It works for any physical product, such as "laptop for coding" or "pram for a newborn"
3. Empty or unclear text gets one clear follow-up question, not an error
4. A request with no product type, such as "something for my mom", gets one follow-up asking what kind of product, with 3 or 4 suggested categories as buttons
5. If the follow-up reply is still unclear or off-topic, the app shows a few example requests and never guesses a category
6. Research never starts until the category is known
7. Each request is saved with a session ID

**US-02: Detect category, use, budget and rules**

Priority: **Must** | Story points: **5** | Sprint: **1** | Tier: **Core** | Requirements: **FR-02**

**As a** shopper, **I want** the app to pick up the category, use, budget and rules I already mentioned, **so that** I am not asked twice.

**Acceptance criteria**

1. "Gaming laptop under 80k" gives category laptop, use gaming, budget up to ₹80,000; k, lakh and ₹ are understood, and ₹ sets the region to India
2. If no budget is stated, none is assumed
3. An ambiguous word such as "watch" makes the app ask which kind before it researches
4. The result is a validated Intent shown back to the user, who can edit it
5. If every LLM provider fails, a keyword rule layer still detects the category and budget

**US-03: Generate category-specific criteria**

Priority: **Must** | Story points: **5** | Sprint: **1** | Tier: **Core** | Requirements: **FR-03, DR-07**

**As a** shopper, **I want** criteria that fit the product I am buying, **so that** the comparison is about what matters for that product.

**Acceptance criteria**

1. The app generates 7 to 10 category-specific criteria and never vague ones like "quality"
2. Value for money is always one of the criteria
3. [DR-07] Criteria change with the intended use, for example gaming versus travel
4. It works for a category it has never seen, and saves the criteria so they are generated once
5. If fewer than 6 come back, it retries once, then uses a fixed fallback set for that kind of product
6. Each criterion has a one-line plain-English definition

**US-04: Smart interview that asks only what matters**

Priority: **Should** | Story points: **8** | Sprint: **4** | Tier: **Core** | Requirements: **FR-04**

**As a** shopper, **I want** to be asked only the questions that matter most for me, **so that** the interview is short.

**Acceptance criteria**

1. Each criterion's weight is a distribution that starts from a default and updates after every answer
2. The next question is about the criterion the app is least sure of that also matters most
3. The interview stops when the top criteria are settled, or after 10 questions at most
4. On 50 simulated shoppers, it matches the fixed questionnaire's accuracy with 30% fewer questions (target)
5. If the model errors, the interview continues in fixed order

**US-05: One question at a time, adapted to earlier answers**

Priority: **Must** | Story points: **8** | Sprint: **1** | Tier: **Core** | Requirements: **FR-05**

**As a** shopper, **I want** questions asked one at a time and adapted to my earlier answers, **so that** it feels like a conversation.

**Acceptance criteria**

1. One question per turn in a chat window, with quick-reply buttons and free text
2. An answer that settles a topic removes related questions; "under 30k" skips premium-tier questions
3. The user can go back and change an answer, and later questions update
4. The first version asks criteria in each category's default importance order; US-04 replaces that ordering
5. Each turn responds within 5 seconds

**US-06: Remember preferences across searches**

Priority: **Should** | Story points: **8** | Sprint: **5** | Tier: **Core** | Requirements: **FR-06**

**As a** returning shopper, **I want** the app to remember what I told it before, **so that** I do not repeat myself.

**Acceptance criteria**

1. Each lasting preference, such as "sensitive to noise" or "never Brand X", is saved with an embedding
2. In a new session, the 5 most similar past preferences above similarity 0.7 pre-fill answers, marked "remembered" and changeable in one click
3. Preferences older than 90 days count for less
4. Preferences never cross categories wrongly; a laptop budget does not fill a headphone budget
5. Repeated questions fall by 50% for users with history (target, measured on 20 seeded sessions)
6. If embeddings fail, the interview runs without pre-filled answers

**US-07: Explain unfamiliar terms on request**

Priority: **Should** | Story points: **2** | Sprint: **6** | Tier: **Core** | Requirements: **FR-07**

**As a** non-technical shopper, **I want** unfamiliar terms explained when I ask, **so that** I can answer questions about things I do not know yet.

**Acceptance criteria**

1. Every question containing a technical term has a "What does this mean?" link
2. Explanations are at most 40 words, in plain language, and say why the term matters for this use
3. Opening an explanation does not count as an answer
4. Definitions come from the saved criteria in US-03, so no extra LLM call is needed

**US-08: Understand rough answers like "medium"**

Priority: **Must** | Story points: **3** | Sprint: **2** | Tier: **Core** | Requirements: **FR-08**

**As a** shopper, **I want** to answer in rough terms like "medium" or "good enough", **so that** I am not forced to invent numbers.

**Acceptance criteria**

1. "Low", "medium" and "high" map to weights 2, 5 and 8 out of 10; "good enough" maps to 3
2. The user can switch to an exact value at any time
3. The mapping is a single table with unit tests, applied the same way everywhere

**US-09: Keep must-haves separate from preferences**

Priority: **Must** | Story points: **5** | Sprint: **3** | Tier: **Core** | Requirements: **FR-09, DR-08**

**As a** shopper, **I want** my must-haves kept separate from my preferences, **so that** a product breaking a must-have is never recommended however good it is otherwise.

**Acceptance criteria**

1. The rubric has two lists: hard rules, and preferences weighted 0 to 10 that add up to 1
2. The user sees which of their statements became hard rules and can switch any of them
3. [DR-08] Hard rules only filter products; preferences only change the order
4. Criteria the interview did not ask about keep the category's default weight
5. A user's manual weight edits survive if the rubric is rebuilt

**US-10: Rule out brands and features**

Priority: **Must** | Story points: **3** | Sprint: **3** | Tier: **Core** | Requirements: **FR-10**

**As a** shopper, **I want** to rule out brands or features I do not want, **so that** I never see them.

**Acceptance criteria**

1. Brand, model or feature exclusions can be typed in words, such as "no Xiaomi" or "not touchscreen"
2. Excluded items never appear in results, and they show in the filtered-out list with the reason
3. An exclusion can be marked "always" and is then saved to memory

**US-11: See and control what the app remembers**

Priority: **Should** | Story points: **3** | Sprint: **6** | Tier: **Core** | Requirements: **FR-22**

**As a** shopper, **I want** to see and control what the app remembers about me, **so that** I stay in charge of my own data.

**Acceptance criteria**

1. The user can view, edit, export as JSON, and delete every stored preference
2. Deleting a preference also deletes its embedding
3. Deleting the account removes all of that user's searches and preferences

**US-39: Understand questions, skips and off-topic replies in the interview**

Priority: **Must** | Story points: **3** | Sprint: **4** | Tier: **Core** | Requirements: **FR-26**

**As a** shopper, **I want** the interview to understand when I ask something back, skip, or change my mind, **so that** it never misreads me.

**Acceptance criteria**

1. Each reply is classified as an answer, a question, an answer with a question, a skip, a correction, off-topic, or unclear
2. A question gets a short answer, then the same interview question is asked again; the answer comes from the criterion's saved definition first, and the LLM only if the definition does not cover it
3. An answer with a question records the answer, then answers the question
4. A skip leaves that criterion at its default weight; a correction updates the earlier answer
5. Off-topic text gets a one-line redirect back to the current question
6. An unclear reply gets one rephrased question with quick-reply buttons, never a guess
7. After two unclear or off-topic replies to the same question, the app skips it, keeps the default weight, and marks it "skipped" so the user can answer later
8. Questions, redirects and skips never count as answers, and the remaining questions continue from where the interview was
9. Reply text is data; nothing typed in a reply can instruct the AI
10. On 70 sample replies (10 per type), at least 90% are classified correctly
11. If the classifier fails, the reply is treated as an answer and the user can correct it

## Epic 2: Multi-Source Research and Retrieval

Find and collect evidence from several sources, and read community posts well.

**US-12: Research several sources in parallel**

Priority: **Must** | Story points: **8** | Sprint: **2** | Tier: **Core** | Requirements: **FR-11**

**As a** shopper, **I want** the app to research several sources, **so that** the answer is not one site's opinion.

**Acceptance criteria**

1. It builds 4 to 6 search queries from the rubric (category, use, budget, region), not from the raw text, and adds a comparison query ("X vs Y") once candidates are known
2. It pulls from at least 3 source types: Reddit threads, web search results and review sites
3. Sources run in parallel, and one failing source never stops the run
4. Near-duplicate threads (more than 60% title overlap) are merged
5. Every page or thread is stored with its URL and fetch time
6. Provider use is counted so the free-tier limits are respected

**US-13: Score Reddit quality and count recommenders exactly**

Priority: **Should** | Story points: **3** | Sprint: **2** | Tier: **Core** | Requirements: **FR-23**

**As a** shopper, **I want** strong Reddit advice separated from noise, **so that** one loud or fake comment does not decide my result.

**Acceptance criteria**

1. Each thread is scored on upvotes, comment count, age and subreddit relevance, and low scorers are dropped
2. Comment trees are read three levels deep; each comment is scored on upvotes, length and whether it names a specific model
3. Distinct recommenders per product are counted with an Aho-Corasick matcher over names and aliases, with no LLM; one user posting ten times counts once
4. Matching respects word boundaries and prefers the longest name, so "1000XM5" inside "WF-1000XM5" is not a separate product
5. Threads lose weight with age, with a half-life of 180 days
6. The same threads always give the same counts

**US-36: Classifier for promotional and duplicate comments**

Priority: **Could** | Story points: **8** | Sprint: **6** | Tier: **Stretch** | Requirements: **FR-23**

**As a** shopper, **I want** promotional and duplicate Reddit comments down-weighted, **so that** shills do not skew the result.

**Acceptance criteria**

1. A classifier trained on 300 hand-labelled comments uses author age, duplicate text and link count
2. It beats the simple rules from US-13 on F1
3. Flagged comments are down-weighted and labelled, never deleted

**US-38: Add YouTube and hard-to-read pages**

Priority: **Must** | Story points: **5** | Sprint: **5** | Tier: **Core** | Requirements: **FR-11**

**As a** shopper, **I want** video reviews and hard-to-read pages included, **so that** the research covers where people actually review products.

**Acceptance criteria**

1. The YouTube Data API finds 3 to 5 relevant review videos per search
2. Transcripts are used only if YouTube's terms allow the chosen method (checked in Sprint 1); otherwise title, description and top comments are used
3. Pages that fail a simple fetch or need JavaScript are read through Jina Reader
4. Video and page text goes through the same extraction and quote rules as every other source
5. If YouTube or Jina fails, the search continues without them and the coverage note says so

**US-42: Read community posts like a person would**

Priority: **Should** | Story points: **5** | Sprint: **5** | Tier: **Plus** | Requirements: **FR-27**

**As a** shopper, **I want** the app to read community posts the way a person would, **so that** sarcasm, weak complaints or one community's taste do not mislead my result.

**Acceptance criteria**

1. Negation words (not, never, barely, far from) flip or weaken the sentiment of their phrase, so "not good for gym use" is never a gym recommendation
2. Common sarcasm markers ("yeah right", "sure, if you say so") reduce the weight of the claim
3. Complaints backed by several independent sources, or naming a concrete failure, count more than a single vague complaint
4. Sentiment per product is compared across communities and labelled consistent, split or single-source
5. A split shows both views with the community each came from; a single-source product gets one step lower confidence
6. The comparison only runs with at least 2 communities and 3 mentions
7. Tested on 40 labelled sentences and 10 products with known community splits

## Epic 3: Evidence, Product Intelligence and Verification

Extract claims, match products, verify, flag conflicts and drop unsourced products.

**US-14: Extract claims with exact quotes**

Priority: **Must** | Story points: **8** | Sprint: **2** | Tier: **Core** | Requirements: **FR-12, DR-05**

**As a** shopper, **I want** claims pulled out of what people wrote, **so that** opinions become comparable evidence.

**Acceptance criteria**

1. The LLM reads each source separately and returns products mentioned, claims (criterion, value or 0 to 10 score, exact quote) and opinions
2. Invalid output is retried once and then discarded, never repaired by guessing
3. Every claim stores its source URL, quote and time
4. [DR-05] Community claims are typed experience; manufacturer and lab claims are typed spec
5. Product types such as "memory foam" are kept apart from buyable products
6. A bad source never stops the run

**US-15: Match product names and variants**

Priority: **Must** | Story points: **8** | Sprint: **3** | Tier: **Core** | Requirements: **FR-13, DR-02, DR-06**

**As a** shopper, **I want** the same product recognised under all its names, **so that** I see one product with all its evidence, not four half-known ones.

**Acceptance criteria**

1. "Sony WH-1000XM5", "XM5" and "1000XM5 headphones" become one product
2. A fine-tuned matching model decides each pair, and only confident matches merge; uncertain pairs stay separate
3. [DR-02] Versions that differ in price or specs, such as RAM, storage or region, are separate variants of one family
4. [DR-06] Aliases are saved and reused in later searches
5. On held-out labelled pairs it beats fuzzy matching, with precision of 0.95 or better (target)
6. If the model fails to load, fuzzy matching runs instead and the run is logged

**US-16: Learned source trust, disputes and unknowns**

Priority: **Must** | Story points: **8** | Sprint: **3** | Tier: **Core** | Requirements: **FR-14, DR-03, DR-09**

**As a** shopper, **I want** to be told when sources disagree, **so that** I can judge a contested fact myself instead of trusting an average.

**Acceptance criteria**

1. Claims for one product and criterion are combined using source weights
2. [DR-03] Manufacturer and lab sources outrank editorial reviews, and editorial outranks community posts
3. Each known site starts from a 0 to 100 trust score adjusted per category, kept in one config table that includes Indian review sites such as 91mobiles, Smartprix, Digit and Gadgets 360
4. Affiliate pages (paths such as /advisor/ or /deals/) start lower than editorial reviews, and Reddit counts as community evidence, never editorial
5. Weights are then learned; sources that often disagree with the wider consensus count for less
6. When independent sources disagree strongly the state is Disputed, and both values are shown with their sources
7. [DR-09] With no trustworthy claim the state is Unknown, and it is never scored as zero
8. It resolves more of the 30 seeded conflicts correctly than majority voting (target)

**US-40: Never recommend an unsourced product**

Priority: **Must** | Story points: **2** | Sprint: **3** | Tier: **Core** | Requirements: **FR-25**

**As a** shopper, **I want** only products that real sources discuss, **so that** the app never recommends something it made up.

**Acceptance criteria**

1. A product with no mention in any source and no source link is dropped before scoring
2. Dropped products are logged with the reason, for testing
3. In recorded runs with planted invented product names, none reaches the results

## Epic 4: Personalized Recommendation and Explainability

Enrich, filter, rank, explain, cite and compare.

**US-17: Filter out products that break the rules**

Priority: **Must** | Story points: **3** | Sprint: **3** | Tier: **Core** | Requirements: **FR-15, DR-01**

**As a** shopper, **I want** products that break my requirements removed before ranking, **so that** every product I see is one I could actually buy.

**Acceptance criteria**

1. [DR-01] Products above the budget, breaking a hard rule, or matching an exclusion are removed in code, never by the LLM
2. Products fall into shown, filtered out with a reason (such as "₹94,990 is above your ₹80,000 budget"), and cannot tell where a hard-rule value is unknown
3. The "cannot tell" group is visible, with a link to research it
4. If nothing passes, the app names the one rule whose relaxation would help most
5. No recommended product breaks a hard rule, checked by tests on every test brief

**US-18: Rank on the user's priorities with coverage**

Priority: **Must** | Story points: **8** | Sprint: **3** | Tier: **Core** | Requirements: **FR-16, DR-09, DR-10**

**As a** shopper, **I want** products ranked on my priorities, **so that** the top result fits me and is not just the most popular.

**Acceptance criteria**

1. Score is the weighted sum of criterion scores using the user's weights
2. [DR-10] Mention counts show how strong the evidence is but never set the score by themselves
3. [DR-09] An unknown criterion takes the average of similar products and is marked estimated
4. Each product shows a data-coverage percentage
5. Labels show best match, runner-up, and the gap to the leader
6. The results page shows ranked cards with score, coverage, main reasons and source links

**US-19: Honest confidence and ties from simulations**

Priority: **Should** | Story points: **5** | Sprint: **4** | Tier: **Core** | Requirements: **FR-16**

**As a** shopper, **I want** to know how sure the app is about the ranking, **so that** I can tell a clear winner from a coin flip.

**Acceptance criteria**

1. 1,000 simulations vary the weights and scores within their uncertainty
2. Each product shows its chance of being first and a high, medium or low confidence label
3. Statistically tied products are marked "too close to call", with the criterion that separates them
4. Rank order stays stable under random 10% weight changes (Spearman 0.9 or better, target)
5. The same inputs and seed always give the same output

**US-20: Explain why a product fits**

Priority: **Must** | Story points: **5** | Sprint: **4** | Tier: **Core** | Requirements: **FR-17**

**As a** shopper, **I want** to see why a product fits me, **so that** I can check the reasoning against my own priorities.

**Acceptance criteria**

1. Each recommended product gets a short "why this fits you" paragraph with its top 3 criteria by contribution and one trade-off
2. The contribution numbers come from code; the LLM only writes the words
3. Anything that could not be verified is named
4. The language is plain, at about school Grade 8
5. If the LLM fails, a template sentence built from the same numbers is shown

**US-21: Click any claim to see its source**

Priority: **Must** | Story points: **3** | Sprint: **3** | Tier: **Core** | Requirements: **FR-18**

**As a** shopper, **I want** to click any claim and see where it came from, **so that** I can check it myself.

**Acceptance criteria**

1. Every factual claim on the results page has a source link and its quote
2. Clicking shows the source name, type, date and exact quote
3. A Disputed claim shows all its sources, not only the winner
4. The quote is stored at retrieval time, so it stays visible even if the page changes

**US-22: Check every explanation sentence against its evidence**

Priority: **Should** | Story points: **5** | Sprint: **5** | Tier: **Core** | Requirements: **FR-21**

**As a** shopper, **I want** unsupported statements removed from explanations, **so that** nothing I read is invented.

**Acceptance criteria**

1. An NLI model checks each explanation sentence against the evidence quotes it cites
2. Unsupported sentences are removed or swapped for a plain template sentence
3. On 100 test pairs with deliberately wrong claims, it catches 80% and wrongly rejects at most 15% of correct ones (target)
4. The check adds no more than 5 seconds
5. If the model fails, only sentences that quote their evidence directly are shown

**US-23: Compare products and re-rank instantly**

Priority: **Must** | Story points: **8** | Sprint: **5** | Tier: **Core** | Requirements: **FR-19**

**As a** shopper, **I want** to compare products and change what matters, **so that** I can explore the trade-off instead of accepting one answer.

**Acceptance criteria**

1. Up to 4 products can be compared side by side on the criteria
2. Moving a weight slider re-ranks in 1 second or less, in the browser, with no new server call
3. Each product shows how far it moved up or down
4. Changing the budget or a hard rule re-filters instantly from the data already loaded

**US-37: Save or share results**

Priority: **Could** | Story points: **3** | Sprint: **6** | Tier: **Plus** | Requirements: **FR-19**

**As a** shopper, **I want** to save or share my results, **so that** I can show them to someone else.

**Acceptance criteria**

1. The results page exports to PDF and CSV
2. A read-only share link opens the same results without needing an account

**US-41: Look up missing facts for the top products**

Priority: **Must** | Story points: **5** | Sprint: **5** | Tier: **Core** | Requirements: **FR-24**

**As a** shopper, **I want** the app to go and find missing facts about the top products, **so that** the recommendation rests on evidence and not on estimates.

**Acceptance criteria**

1. After a first ranking, the top 5 products' highest-weight criteria with no evidence trigger a targeted lookup
2. At most 6 searches and 1 batched LLM call per search, cached for 7 days
3. A value is kept only when the retrieved page actually supports it, and its source is cited
4. Products are then re-scored, and only what is still missing is estimated
5. If the lookup fails, the stage is skipped and values stay estimated
6. Data coverage of the top 5 is measured before and after on 10 briefs

## Epic 5: Commerce and Purchase Discovery

Price, availability and buy links for the exact model.

**US-24: Current price and buy link for the exact model**

Priority: **Must** | Story points: **5** | Sprint: **5** | Tier: **Core** | Requirements: **FR-20, DR-01, DR-04**

**As a** shopper, **I want** the current price and where to buy the exact model, **so that** I do not click through and find a different product or price.

**Acceptance criteria**

1. It finds price, store, availability and link for the top products using Serper shopping results, with retailer lookups run in parallel
2. A link is accepted only if its title matches the model name, and the variant where the data allows
3. [DR-04] Every price shows an as-of time, and prices older than 24 hours are labelled "may have changed"
4. [DR-01] Offers above the budget are flagged
5. If no price is found, the card says so and links to a store search
6. Links open the store's own page; the app does not check out for the user

## Epic 6: Platform Engineering, Security and Reliability

A skeleton that always runs: secure, reliable, fast, modular and tested.

**US-25: Repository, CI and secret scanning**

Priority: **Must** | Story points: **3** | Sprint: **1** | Tier: **Core** | Requirements: **NFR-08**

**As a** developer, **I want** a clean repository with automatic checks, **so that** the team can work safely in parallel and no key leaks.

**Acceptance criteria**

1. One GitHub repo with all 10 members, a protected main, and one review required per merge
2. CI runs ruff, mypy, ESLint and the tests on every pull request
3. Secret scanning is on, .env is ignored, and keys live only in GitHub Secrets
4. The database schema lives in versioned migration files from the first table onward
5. Slack channels and the Projects board are created, and the folder layout from the plan exists

**US-26: One LLM wrapper with fallback and context fitting**

Priority: **Must** | Story points: **5** | Sprint: **1** | Tier: **Core** | Requirements: **NFR-04, NFR-05**

**As a** developer, **I want** one LLM wrapper with fallback and output checks, **so that** one provider failing does not stop the app.

**Acceptance criteria**

1. Every LLM call uses one function that takes a task, a prompt and an expected schema
2. Before each call it counts the input tokens and checks them against the chosen model's context window, leaving room for the answer plus a 10% margin
3. Input that does not fit is trimmed (lowest-ranked comments first) or split into chunks whose results are merged; nothing is ever sent over the limit
4. Each model's context window and output limit live in one config file, checked by the API smoke test
5. When a call falls back to a model with a smaller window, the input is re-fitted to that model's limit first
6. On rate limits, timeouts or server errors it moves through Groq, Gemini, OpenRouter and Cerebras
7. Output is validated against the schema; invalid output is retried once, then goes to the next provider, then to a safe default
8. Each call logs its provider, tokens, time and outcome
9. Tasks tagged personal are never sent to the Gemini free tier, and a test proves it
10. A test sends an oversized input and checks it is trimmed or split, never rejected by the provider
11. A smoke test of every API records its real limits

**US-27: Fetch sources in parallel**

Priority: **Must** | Story points: **2** | Sprint: **2** | Tier: **Core** | Requirements: **NFR-02**

**As an** ML engineer, **I want** independent fetches to run in parallel, **so that** research time is set by the slowest source and not the sum.

**Acceptance criteria**

1. Source fetches run in a thread pool with a fixed size
2. Total fetch time is close to the slowest source
3. Each API's rate limit is respected

**US-28: Treat retrieved content as untrusted**

Priority: **Must** | Story points: **3** | Sprint: **2** | Tier: **Core** | Requirements: **NFR-09**

**As a** security representative, **I want** retrieved content treated as untrusted, **so that** a hostile web page cannot steer the recommendation.

**Acceptance criteria**

1. Web, Reddit and video text is passed to the LLM as labelled data, with an instruction to ignore any commands inside it
2. Extraction output is accepted only as schema JSON
3. HTML is stripped and text length is capped
4. 10 test pages with hidden instructions such as "rank product X first" do not change any result

**US-29: Trace every claim to its evidence**

Priority: **Must** | Story points: **3** | Sprint: **6** | Tier: **Core** | Requirements: **NFR-14**

**As a** shopper, **I want** every claim and decision traceable to its evidence, **so that** nothing in the results is unexplained.

**Acceptance criteria**

1. Every claim and ranking decision is stored with its source ID, quote and time
2. A saved result reopens with the same evidence
3. An automated test picks random claims from recorded results and finds each one's source in the database

**US-30: Login and private preferences**

Priority: **Must** | Story points: **5** | Sprint: **4** | Tier: **Core** | Requirements: **NFR-06, NFR-07**

**As a** shopper, **I want** my saved preferences protected, **so that** nobody else can see what I am shopping for.

**Acceptance criteria**

1. Login uses Supabase Auth with Google sign-in
2. Preferences and past searches are visible only to their owner, enforced by database row-level security
3. Test: user A cannot read user B's data through the API
4. Guests can search without signing in, but nothing of theirs is saved

**US-31: Handle outside failures gracefully**

Priority: **Must** | Story points: **5** | Sprint: **4** | Tier: **Core** | Requirements: **NFR-03**

**As a** system administrator, **I want** outside failures handled gracefully, **so that** one outage does not take the app down.

**Acceptance criteria**

1. Every external call has a timeout from the stage contract table and one retry
2. If one source type fails, results still arrive with a lower-coverage note
3. If every LLM provider fails, the user sees a clear message and a retry button, not a crash
4. A site that keeps failing is skipped for 24 hours, then tried again; a rate limit counts less than a block
5. Tests inject failures for each provider and source

**US-32: Core logic testable offline**

Priority: **Must** | Story points: **3** | Sprint: **2** | Tier: **Core** | Requirements: **NFR-15**

**As a** developer, **I want** the core logic testable without the internet, **so that** tests are fast and never use up API quota.

**Acceptance criteria**

1. Filtering, scoring, ranking, weight mapping and mention counting run in tests with no network
2. Tests use recorded API responses
3. Coverage is at least 70% on core modules and 80% on decide/, and the fast suite runs in under 60 seconds
4. CI blocks a merge if tests fail

**US-33: Plain language and accessible controls**

Priority: **Must** | Story points: **5** | Sprint: **6** | Tier: **Core** | Requirements: **NFR-10, NFR-11**

**As a** shopper using assistive technology or with no technical background, **I want** plain language and accessible controls, **so that** I can use the app on my own.

**Acceptance criteria**

1. Questions, explanations and results are at about Grade 8 reading level, and 4 of 5 non-technical testers understand a result without help
2. The interview, sliders and compare view work with the keyboard alone
3. Colour contrast passes WCAG AA, and every slider has a number box
4. An automated accessibility check scores 90 or more

**US-34: Fast responses and exact-match cache**

Priority: **Should** | Story points: **5** | Sprint: **6** | Tier: **Core** | Requirements: **NFR-01, FR-28**

**As a** shopper, **I want** quick responses, **so that** I do not give up halfway through.

**Acceptance criteria**

1. 90% of interview turns finish within 5 seconds
2. A full search finishes within 120 seconds and shows live progress; none runs past 150 seconds
3. Repeating a search within 24 hours returns from cache in 3 seconds or less
4. A cached result is reused only when the request, category and priorities match (and, for signed-in users, their stored preferences); a result scored for different priorities is never reused
5. Measured on 20 test briefs, with the slowest stage named

**US-35: Modules with clear interfaces**

Priority: **Should** | Story points: **3** | Sprint: **6** | Tier: **Core** | Requirements: **NFR-12, NFR-13**

**As a** developer, **I want** modules with clear interfaces, **so that** categories, sources and providers can change without touching the core.

**Acceptance criteria**

1. A brand-new category works end to end with no code change
2. An LLM or search provider can be swapped by configuration only
3. Each module has a documented input and output, and one integration test per module boundary

**US-46: Circuit breaker and provider rotation**

Priority: **Must** | Story points: **4** | Sprint: **2** | Tier: **Core** | Requirements: **NFR-04**

**As a** developer, **I want** failing providers switched off automatically and free limits used efficiently, **so that** the app stays fast and inside quota.

**Acceptance criteria**

1. A circuit breaker trips a provider after repeated failures in a short window; rate limits and server errors cool it down for 60 to 120 seconds, and a bad key marks it dead for the run
2. Bulk extraction rotates across providers in a round-robin pool
3. Each provider has a daily token counter, and calls stop before the budget is exceeded
4. Tests simulate a failing, a slow and a rate-limited provider

**US-47: Per-user rate limits**

Priority: **Should** | Story points: **2** | Sprint: **6** | Tier: **Plus** | Requirements: **NFR-16**

**As a** system administrator, **I want** limits on how often each person can search, **so that** one user cannot use up the whole free quota.

**Acceptance criteria**

1. 10 searches a minute per signed-in user or per network address for guests, and 200 requests a minute overall
2. Over the limit, the user sees when they can try again

**US-48: Golden tests for broken LLM output**

Priority: **Must** | Story points: **3** | Sprint: **2** | Tier: **Core** | Requirements: **NFR-05**

**As a** developer, **I want** every parser proven safe against broken LLM output, **so that** a provider changing its format fails the build instead of corrupting a ranking.

**Acceptance criteria**

1. About 20 malformed outputs per parser - intent, criteria, interview reply, rubric, extraction, enrichment and explanation
2. Cases cover fenced JSON, missing keys, wrong types, out-of-range values, trailing commas, cut-off text, plain prose, extra invented fields and embedded instructions
3. Every case produces the canonical shape or a clean fallback, and none crashes
4. New parsers must add their cases before merging

**US-49: Recorded-run replay and the CI quality gate**

Priority: **Must** | Story points: **5** | Sprint: **4** | Tier: **Core** | Requirements: **NFR-17**

**As a** developer, **I want** real pipeline output tested and a quality gate on every change, **so that** a model update or a code change can never quietly make results worse.

**Acceptance criteria**

1. A capped recorder captures 5 real runs (sources, LLM outputs, final results), at most twice a sprint
2. CI replays them offline on every pull request - at least 90% of expected products survive extraction, every claim has a quote, and no product lacks a source
3. thresholds.yaml holds a floor for every metric; CI fails if a metric drops below its floor or falls more than 10 points from main
4. A validator recomputes each test scenario's expected winner with the scoring code and fails if a hand label disagrees
5. The fast gate finishes in under 3 minutes; the full evaluation runs nightly

**US-50: Worst-case test suite**

Priority: **Must** | Story points: **5** | Sprint: **6** | Tier: **Core** | Requirements: **NFR-03**

**As a** system administrator, **I want** every failure the app might meet tested, **so that** it works in the worst case and not just the demo case.

**Acceptance criteria**

1. One automated fault test per row of the worst-case table in the plan
2. A Playwright browser test runs request, interview and results on every merge to main
3. A written drill script lets anyone break a key, a source or the database live and check the result
4. All fault tests pass before the code freeze

**US-51: Measure consistency, personalisation and popularity bias**

Priority: **Should** | Story points: **5** | Sprint: **6** | Tier: **Core** | Requirements: **NFR-17, DR-10**

**As a** developer, **I want** the recommendations themselves measured, **so that** we can show they are consistent, personal and not just popular.

**Acceptance criteria**

1. Paraphrase consistency - in 10 pairs of reworded queries, the top pick matches in at least 8
2. Personalization strength - 5 personas on the same brief produce at least 3 different top picks
3. Popularity bias - in scenarios where the most-mentioned product fits worst, it never ranks first
4. Every metric is written to one generated results report used in the final submission

**US-52: Walking skeleton, pipeline runner and CLI**

Priority: **Must** | Story points: **5** | Sprint: **1** | Tier: **Core** | Requirements: **NFR-12**

**As a** developer, **I want** the whole pipeline running end to end from the first sprint, **so that** every later feature improves a working app instead of completing a broken one.

**Acceptance criteria**

1. Every stage has its contract model and a simple stub implementation
2. The pipeline runner calls the stages in order, applies each stage's time limit and fallback, and stops at 150 seconds with the best result so far
3. `python -m app.cli "best laptop for coding under 70k"` prints a ranked list with sources
4. The same pipeline feeds a plain results page in the browser
5. The pipeline finishes without the database and saves at the end
6. Each stage has a contract test, and replacing any stage with one that fails triggers its fallback

**US-53: Progress survives a dropped connection**

Priority: **Could** | Story points: **2** | Sprint: **6** | Tier: **Plus** | Requirements: **NFR-03**

**As a** shopper, **I want** the live progress to survive a dropped connection, **so that** I do not lose a two-minute search.

**Acceptance criteria**

1. The progress stream reconnects automatically with increasing waits
2. Progress resumes from the last finished stage
3. With no update for 60 seconds, the page says so and offers to reload the result

**US-54: Team diagnostics page**

Priority: **Could** | Story points: **3** | Sprint: **6** | Tier: **Plus** | Requirements: **NFR-03**

**As a** developer, **I want** one page showing how the system is behaving, **so that** we can find problems fast during testing and the demo.

**Acceptance criteria**

1. A team-only page shows provider status, recent LLM calls, stage times, fallbacks used and coverage per search
2. It is behind the team's login and hidden from other users